

DNS Domain Name System

Warum DNS?

- Da sich der Mensch sich IP-Adressen z.B. für Webseiten die er besuchen möchte nicht gut merken kann, werden für die **Navigation im Internet Namen** verwendet
- Da ein PC allerdings zum **Kontaktieren eines Servers** allerdings eine **IP-Adresse** benötigt wurde das Domain Name System entwickelt.
- Das DNS sorgt für die **automatische Auflösung von Namen in IP-Adressen**
- Das DNS ist vergleichbar mit der Telefonauskunft.

DNS Domain Name System

Die Anfänge

- Zu Beginn des Internets wurde diese **Zuordnung über eine Textdatei** bewerkstelligt. Diese Datei existiert auch heute noch
- Unter Windows: C:\Windows\System32\drivers\etc
- Unter Linux: /etc/hosts

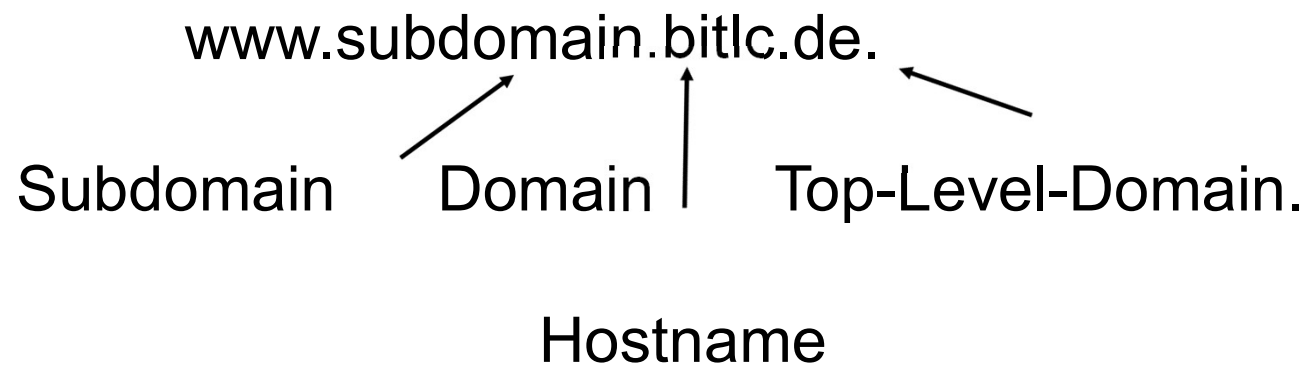
DNS Domain Name System

Die Weiterentwicklung

- Das DNS besitzt einen **hierarchischen Aufbau** sowohl für die **Struktur der Namen** als auch die **Serverstruktur**.
- Durch das rasante **Wachstum des Internets** wurde die **zentrale Auflösung** durch die Hosts-Datei immer **schwieriger**.
- Deshalb wurde eine **dezentrale Serverstruktur** implementiert, bei der die **Server weltweit verteilt** sind.

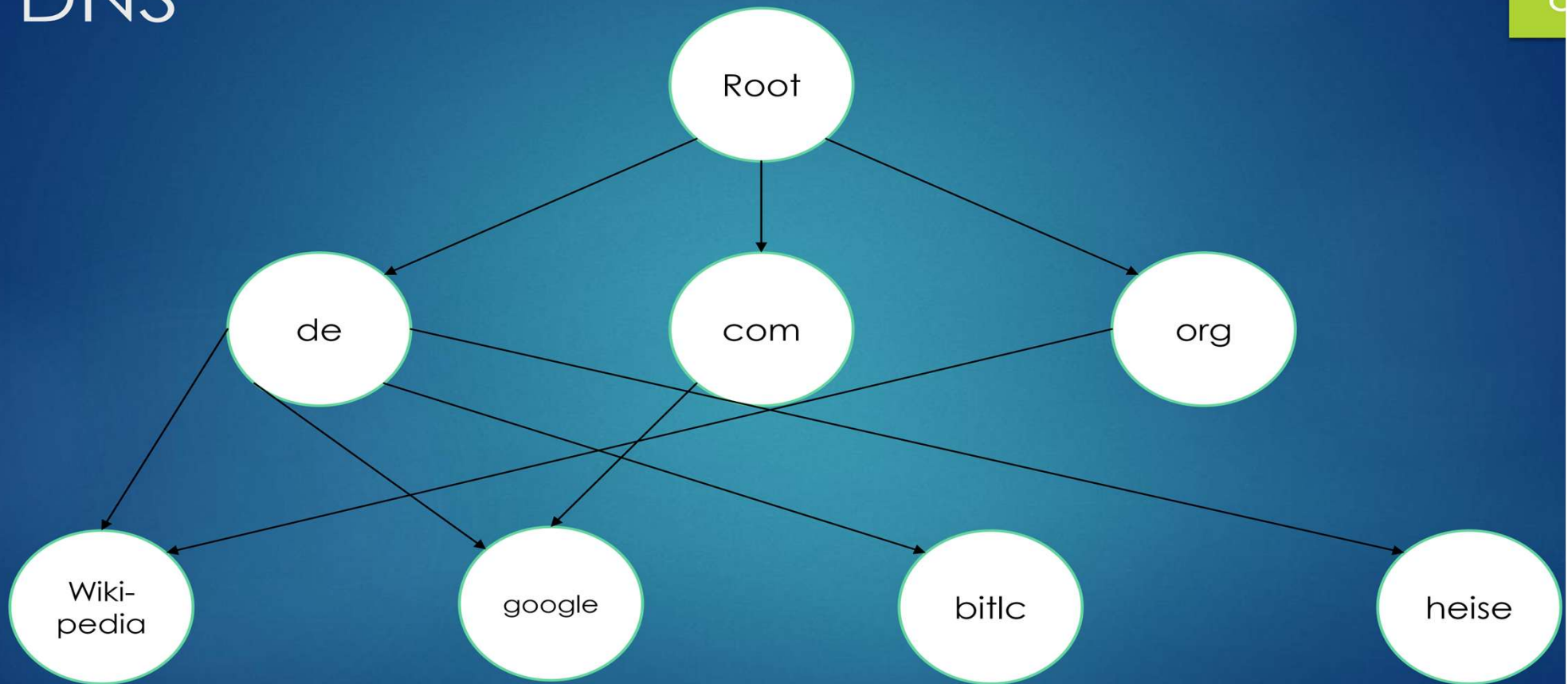
DNS Domain Name System

Der vollständige Name eines Servers wird **Fully Qualified Domain Name FQDN** genannt.



DNS Domain Name System

DNS



DNS Domain Name System

Beteiligte Komponenten

DNS-Server:

Diese Server sorgen für die eigentliche Auflösung von Namen in IP-Adressen und andersherum.

Namensraum:

Namensraum ist eine Domain für den ein bestimmter DNS-Server autoritativ zuständig ist.

Resolver:

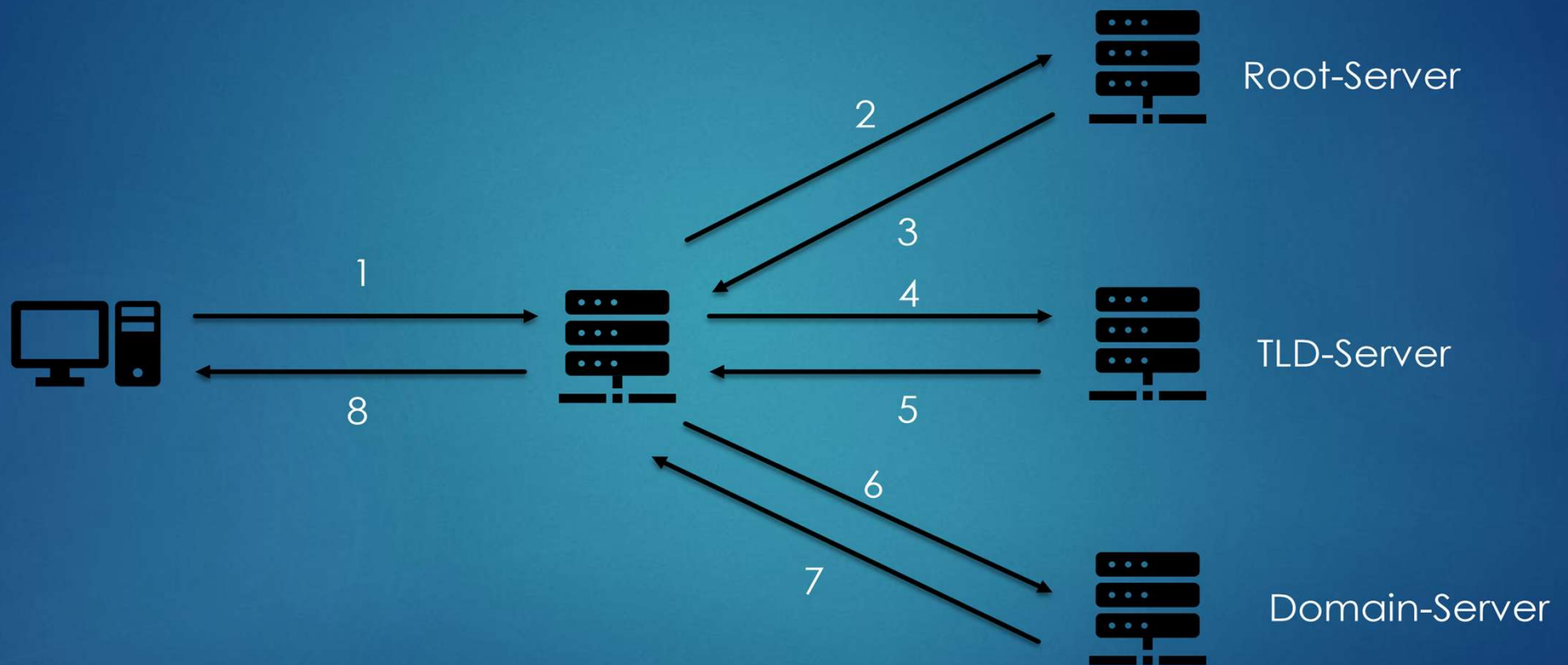
Der Resolver ist eine Software, die meist teil eines Betriebssystems ist. Der Resolver nimmt anfragen anderer Programme entgegen und leitet sie an die DNS-Server weiter. Konfigurationsdatei ist `/etc/resolv.conf`

DNS Domain Name System

Da das DNS hierarchisch aufgebaut ist hat nicht jeder DNS-Server die IP zu jedem Server.

- Deshalb erfolgt eine **DNS-Anfrage meist in mehreren Schritten**.
- Ein Client stellt an **i.d.R eine rekursive Anfrage**. Der angesprochene Server ist dann für die komplette Namensauflösung zuständig.
- Bei einer iterativen Anfrage teilt der angesprochene DNS-Server nur die Adresse des Servers mit, der als nächstes angefragt werden muss.

DNS Domain Name System



DNS Domain Name System

1. Der Client stellt eine **rekursive Anfrage** an den konfigurierten **DNS-Server**.
2. Der **DNS-Server** kann die Adresse selber nicht auflösen und stellt **eine Reihe von iterativen Anfragen**. Die **erste** ist an einen **Root-Server** gerichtet.
3. Der **Root-Server** gibt als Antwort die **IP-Adresse** des zuständigen **Top-Level-Domain-Servers** aus.
4. Der **DNS-Server** stellt eine weitere **Anfrage** an den **TLD-Server**.

DNS Domain Name System

Fortsetzung

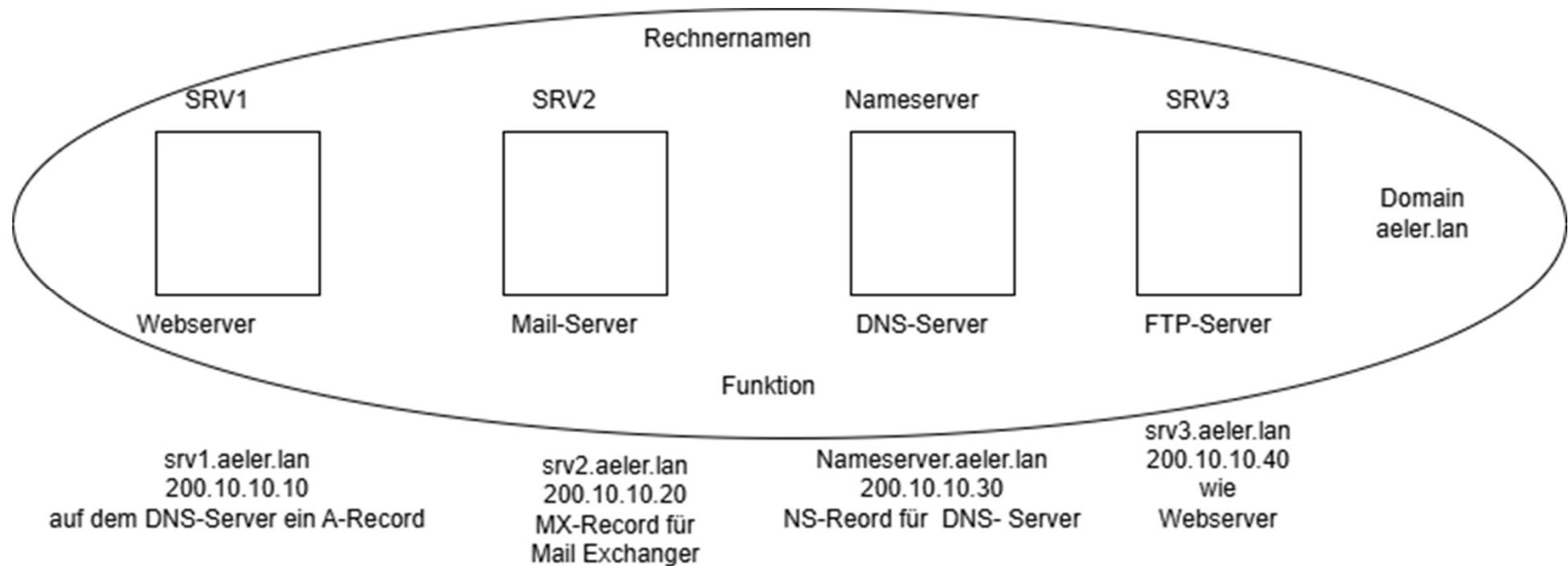
5. Der **TLD-Server** antwortet mit der **IP-Adresse** des zuständigen **Domain-DNS-Servers**
6. Der **DNS-Server** stellt eine Anfrage an den **Domain-DNS-Server**, dessen IP er vom TLD-Server erhalten hat.
7. Der **Domain-DNS-Server** gibt als **Antwort die IP-Adresse** des angefragten **Hosts** raus.
8. Der **DNS-Server** liefert die **Host-IP-Adresse** an den Client zurück

DNS Domain Name System

DNS-Testwerkzeuge

1. dig : gibt einen vollständigen Überblick über Namensauflösung mit Metadaten (z.B Zeitstempel, flags usw.)
z.B dig www.google.de
2. nslookup: nur IP-Adressen und Namen, IP-Adressen der DNS-Server
z.B nslookup www.google.de
3. host : für eine schnelle kurze Antwort, IP-Adressen des Ziels

DNS Domain Name System



DNS Domain Name System

DNS-Tabelle, Beispiele mit verschiedenen Record-Typen

Name	Type	IP-Adresse
srv1.aeler.lan	A	200.10.10.10
srv2.aeler.lan	MX	200.10.10.20
Nameserver.aeler.lan	NS	200.10.10.30
srv3.aeler.lan	A od. AAAA (IPv6)	200.10.10.40

DNS Domain Name System

1. A-Record (Address Record)

Was er macht: Er verbindet einen Hostnamen direkt mit einer IPv4-Adresse.

Wozu er dient: Das ist das direkte digitale Gegenstück zu der /etc/hosts. Wenn ein Client srv1.aeler.lan aufruft, liefert der A-Record die IP 200.10.10.10 zurück. Jedes Mal, wenn ein Rechner im Netz unter seinem eigenen Namen per IPv4 erreichbar sein soll, benötigt er einen A-Record.

2. AAAA-Record (IPv6 Address Record)

Was er macht: Genau dasselbe wie der A-Record, aber er verbindet den Hostnamen mit einer IPv6-Adresse.

Wozu er dient: Da IPv4-Adressen knapp sind, läuft das moderne Internet nativ auf IPv6. Die vier „A“ stehen symbolisch dafür, dass eine IPv6-Adresse viermal so lang (128 Bit) ist wie eine IPv4-Adresse (32 Bit). Im Labor nutzt ihr ihn, sobald Postfix oder andere Dienste über [::1] oder globale IPv6-Schnittstellen kommunizieren.

DNS Domain Name System

3. MX-Record (Mail Exchanger)

Was er macht: Er zeigt exakt auf den zuständigen Mailserver einer Domain.

Wozu er dient: Wenn jemand eine E-Mail an root@aeler.lan schickt, weiß der absendende Postfix-Server nicht, welcher PC im Netzwerk die Post annimmt. Er fragt das DNS nach dem MX-Record für @ (die Domain). Das DNS antwortet: „Der Postbote für die gesamte Domain ist srv2.aeler.lan“.

Die Priorität: Dem MX-Record wird immer eine Zahl als Priorität vorangestellt (z. B. 10 oder 20). Habt ihr zwei Mailserver, versucht der Absender immer zuerst den Server mit der niedrigsten Zahl. Schlägt das fehl, weicht er auf die höhere Zahl (Backup-Mailserver) aus.

4. CNAME-Record (Canonical Name Record)

Was er macht: Er erstellt einen Alias (einen Spitznamen) für einen bereits existierenden Namen.

Wozu er dient: Er verbindet einen Namen nicht mit einer IP, sondern mit einem anderen Namen. Wenn srv1.aeler.lan unser primärer Webserver ist, möchten wir, dass die User ihn auch über www.aeler.lan erreichen. Du sagst im CNAME: www verweist auf srv1.aeler.lan.

Der unschlagbare Vorteil: Ändert sich die IP-Adresse des Servers, muss man im DNS nur den einen A-Record von srv1 anpassen. Der www-Eintrag wandert automatisch mit, ohne dass man dort eine IP ändern muss.

DNS Domain Name System

5. NS-Record (Name Server Record)

Was er macht: Er gibt an, welche DNS-Server autoritativ für diese Zone zuständig sind.

Wozu er dient: DNS ist hierarchisch aufgebaut. Wenn ein Root- oder TLD-Server eine Anfrage delegiert, nutzt er den NS-Record, um dem anfragenden Server zu sagen: „Ich kenne die Antwort nicht, aber frage den Server nameserver.aeler.lan, der verwaltet diese Domain hieb- und stichfest!“.

6. PTR-Record (Pointer Record – "Reverse DNS")

Was er macht: Er macht genau das Gegenteil des A-Records. Er übersetzt eine IP-Adresse rückwärts in einen Namen.

Wozu er dient: Das ist für euren Mailserver-Workshop ein absoluter Pflichtbegriff! Große E-Mail-Anbieter (wie Gmail oder GMX) nehmen heute oft keine E-Mails mehr an, wenn der PTR-Record nicht stimmt. Wenn unser Server Mails sendet, sieht der Empfänger die IP. Er macht eine Reverse-Anfrage ans DNS: „Gehört diese IP wirklich zu dem Mailserver, als der er sich ausgibt?“ Stimmen A-Record und PTR-Record nicht überein, fliegt die Mail sofort in den Spam-Ordner.

DNS Domain Name System

Primary-DNS-Server

Diese Server sind für einen bestimmten Namensraum autoritativ zuständig. Nur auf einem Primary-Server können Einträge geändert oder hinzugefügt werden.

Secondary-DNS-Server

Diese Server kopieren die Einträge von einem Primary-DNS-Server. Sie werden als Backup-Server eingerichtet oder zum Lastenausgleich.

Cache-Only-Server

Diese Server führen keine eigenen Tabellen sondern dienen als Zwischenspeicher für erfolgte Anfragen.

Alle haben, wie beim IPv4-Header, eine TTL, die der DNS-Antwort eine Haltbarkeit in Sekunden mitgibt.

DNS Domain Name System

Primary-DNS-Server

Diese Server sind für einen bestimmten Namensraum autoritativ zuständig. Nur auf einem Primary-Server können Einträge geändert oder hinzugefügt werden.

Secondary-DNS-Server

Diese Server kopieren die Einträge von einem Primary-DNS-Server. Sie werden als Backup-Server eingerichtet oder zum Lastenausgleich.

Cache-Only-Server

Diese Server führen keine eigenen Tabellen sondern dienen als Zwischenspeicher für erfolgte Anfragen.

Alle haben, wie beim IPv4-Header, eine TTL, die der DNS-Antwort eine Haltbarkeit in Sekunden mitgibt.

DNS Workshop, Basis www.server-world.info

Namensauflösung mit dnsmasq und BIND9 unter Rocky Linux

Ziel:

- Ablösung der manuellen /etc/hosts-Konfiguration durch zentrale Dienste
- Zuweisung der Rollen im Netzwerk.

DNS

Der Server-World-Ansatz – Zwei Wege zum Ziel

Weg1: dnsmasq

- Kombiniert einen leichtgewichtigen DNS-Forwarder und DHCP-Server.
- Optimal für lokale Testumgebungen und schnelle Lernerfolge.
- Der Clou: Nutzt die vorhandene /etc/hosts des Routers als direkte Datenbasis für das Netzwerk.

DNS

Die neue Netz-Architektur (Wer fragt wen?)

Zentrale DNS-Weiche auf dem Router:

- Die Clients tragen ab sofort nur noch die Gateway-IP des Routers in ihrer /etc/resolv.conf ein.
- Die fehleranfälligen lokalen /etc/hosts-Einträge auf den Clients werden komplett gelöscht.

Der intelligente Datenfluss bei einer Anfrage:

- 1. Lokaler Name (z. B. rocky1A.NetzA.local):
Router kennt den Namen, fängt die Anfrage ab und antwortet sofort lokal.
- 2. Internet-Name (z. B. www.google.de):
Router weiß, dass er nicht zuständig ist, und leitet die Anfrage (Forwarding) an den Schulungsraum-Server weiter.

DNS

Praxis-Schritt 1: dnsmasq auf dem Router einrichten

Installation & Aktivierung:

- Mit dnf installieren wir das Paket auf dem Rocky-Router und aktivieren es als dauerhaften Systemdienst.

Wichtige Parameter in der Konfiguration:

- **interface:** Bestimmt, auf welcher Netzwerkkarte der Router auf Client-Anfragen lauscht.
- **server:** Definiert die "Upstream"-DNS-Server für die Weiterleitung ins Internet.
- **domain:** Weist dem Labornetzwerk die lokale Domain zu.

1. Paket installieren

```
sudo dnf install -y dnsmasq
```

2. Minimale /etc/dnsmasq.conf

```
interface=ens256
```

DNS-Forwarding (Schulungsnetz & Backup)

```
server=172.21.0.9
```

```
server=1.1.1.3
```

Lokale Domain-Zuweisung erzwingen

```
expand-hosts
```

```
domain=netzA.local
```

3. Dienst aktivieren und starten

```
sudo systemctl enable --now  
dnsmasq
```

DNS

Das Herzstück: Die Datei /etc/hosts auf dem Router

- **Wie dnsmasq Daten liest:**
Der Dienst nutzt standardmäßig die lokale Host-Datei des Routers. Er wandelt diese Einträge dynamisch in echte DNS-Antworten für die Clients um.
- **Der "Alias"-Trick für den Mailserver:**
Werden hinter einer IP-Adresse mehrere Namen durch Leerzeichen getrennt angegeben, interpretiert das DNS diese automatisch als vollwertige, zusätzliche Namen (Aliase) für diesen Host. (Optional)

Vorlage für die /etc/hosts:

- siehe Tag 31 hosts_vorlage.txt

DNS

Praxis-Schritt 2: Die Clients umstellen

- **Konfiguration des Stub-Resolvers:**
Damit die VMs (rocky1A, mint1A usw.) das neue DNS nutzen, wird ihre /etc/resolv.conf angepasst. Sie darf nur noch auf das Router-Gateway zeigen.
- **Die alten Labor-Einträge in den Client-/etc/hosts-Dateien müssen gelöscht werden. Nur so beweisen wir gleich im Test, dass die Auflösung wirklich über das Netzwerk läuft!**

1. Datei: /etc/resolv.conf auf den Clients:

nameserver 172.16.7.33 # Die jeweilige Router-IP
search netzA.local

2. Datei: /etc/hosts auf den Clients bereinigen

127.0.0.1 localhost
::1 localhost

HINWEIS: Alle anderen Labor-Einträge löschen!

DNS

DNS-Überprüfung im Labor (Troubleshooting)

- **Werkzeuge zur Verifizierung:**
Wir nutzen Standard-Utilities, um die Namensauflösung auf den Clients live zu testen.

Der entscheidende Kontrollblick bei dig:

- In der Ausgabe des dig-Befehls müssen wir ganz unten auf die Zeile **SERVER:** achten.
- Hier muss zwingend die IP-Adresse des Routers (172.16.7.33) stehen – das ist der Beweis, dass euer lokaler DNS-Dienst geantwortet hat!

1. Schneller Check mit nslookup
nslookup rocky2B.netzB.local

2. Detaillierter Blick via dig
dig rocky2B.netzB.local
Optional wenn da????:

3. Prüfung des Mailserver-Alias
dig mx.netzA.local

4. Internet-Gegentest (Prüfung Forwarding)
dig www.google.com

DNS

Der Übergang zum Enterprise-Standard BIND9

Was sich im hier ändert:

Keine /etc/hosts mehr:

- Wir schalten die Nutzung der einfachen Texttabelle auf dem Router ab.

Echte Zonendateien:

- Wir schreiben eigene Forward-Zonen (Name $\rightarrow$ IP) und Reverse-Zonen (IP $\rightarrow$ Name) von Hand.

Offizielle Resource Records:

- Wir deklarieren explizite Einträge vom Typ A, AAAA, CNAME und NS.

Vorbereitung für Mailserver:

- Wir binden echte MX (Mail Exchanger) Records inklusive ihrer Prioritäten ein.